Requirement 1

Design 1

Create class:

1. TeleportCube
2. TeleportAction
3. TeleDoor
4. TeleportationCircle
5. Fire
6. Dirt
7. Burning
8. Earth

Pro	**Cons**
Explicit, easy to read and understand for new programmers (not required to move too many classes to understand the code)	Duplication of code, such as TeleDoor and TeleportationCircle share a lot of teleport behaviour but are separate.
	TeleportAction might end up full of if/else or instanceof checks to distinguish types.
	Harder to extend → adding a new teleport type requires modifying TeleportAction.

Design 2

Create new class/interface

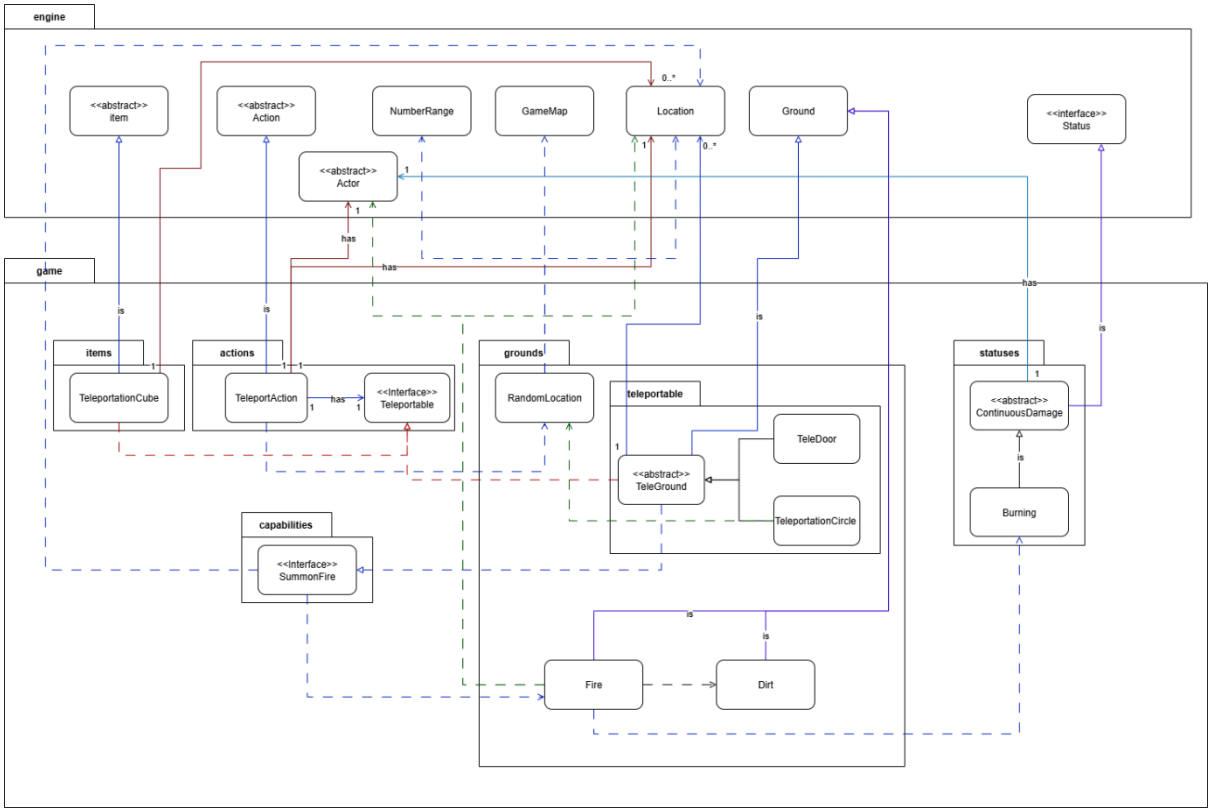
1. Abstract TeleGround
2. Teleportable interface
3. Abstract ContinuousDamage
4. SummonFire interface
5. RandomLocation concrete Class

Pro	**Cons**
Polymorphism, TeleportAction can now work on any Teleportable no need for instanceOf/ if/else	Slightly more abstract
Enhance open close principle, new teleport types only require implementing Teleportable and its logic, no change in existing code (e.g., TeleportAction)	May introduce extra inheritance.
DRY principle, by allocate share logic for both TeleportDoor & TeleportationCircle	Must follow the contract, (e.g., Precondition, Postcondition etc...)

into abstract TeleGround, also ContinuousDamage handle the basic logic for continuous damage to an actor so Burning can just extend and use it future can be easier to be extend without violating DRY (e.g., Bleeding, Poison etc....).	
With SummonFire as an injector interface take in a Location so that in future other class that can summon fire will just implement it satisfying OCP where RandomLocation takes in a map get a random location within the map, so TeleportAction just uses the class to get random location as the same reasoning above it satisfy OCP where other classes may also need a random location.	

Choose:

Design 2, compare to Design 1 it was more abstract but provides cleaner, more maintainable, and extensible code by following OCP, DRY, and polymorphism **meanwhile** ensure no violation on SOLID principles and highly adhere to OOP. Overall, while Design 1 provides simplicity and a direct mapping to game concepts, Design 2's advantages in maintainability, scalability, and long-term sustainability outweigh the cons. Therefore, **Design 2 is the more worthwhile choice**.



(There are some quality effect on this picture for more clarity please refer to the git)

1. What classes will exist in the extended system
    a. Interface:
        i. Teleportable
        ii. SummonFire
    b. Abstract class:
        i. TeleGround
        ii. ContinuousDamage
    c. Concrete Class:
        i. RandomLocation
        ii. TeleDoor
        iii. TeleportationCircle
        iv. TeleportCube
        v. TeleportAction
        vi. Fire
        vii. Dirt
        viii. Burning
        ix. Earth
2. Roles & Responsibilities
    a. Teleportable → Defines teleport behaviour generically & represents any object that can be used to teleport in the system

   b. SummonFire → Defines summon fire behaviour to a specific location burnLocation().
   c. TeleGround → Holds common ground-based teleport logic extends Ground while implement Teleportable.
      i. TeleDoor → Handles burning destination surroundings and extends TeleGround
      ii. TeleportationCircle → Burns one random source tile and extends TeleGround.
   d. TeleportCube → Handle the details and attributes for itself extends from Item while implement Teleportable.
   e. RandomLocation → Return a random location within a map.
   f. TeleportAction → Executes teleport by calling teleportTo on any Teleportable object, meanwhile checking the % of successful rate if fail then perform malfunction by randomly teleport to a position within the map (Using RandomLocation class method), extends from Action.
   g. Fire → Ground hazard that damages actors and transform into Dirt after 3 turns, extend from Ground.
   h. Dirt → Represent permanently burned terrain in the system, extends from Ground, used by Fire class (for now).
   i. ContinuousDamage → Provides reusable structure for status effects with turn-based ticking damage, implement Status.
   j. Burning → Implementation of ContinuousDamage (5 damage/turn for 5 turn, stackable etc... by passing info to it), extends ContinuousDamage.
   k. Earth → Act as injector class inject type of door into the system (e.g. map)
3. How these classes relate to and interact with the existing system.
   a. Earth class inject TeleportCube, TeleDoor, TeleportationCircle into existing map/ actor inventory as new interactive objects, itself, handle where can be teleported to once created, and has allowableActions for getting TeleportAction for each destination, TeleportCube is extend from Item class and remaining extend from TeleGround (which extend Ground).
   b. TeleportAction take in a location and integrates with the game (e.g., player choose to teleport to a certain place) it extends from Action.
   c. Fire and Dirt integrate with the map terrain system (extend Ground) and often use by interface SummonFire which implemented by TeleGround etc....
   d. Burning integrates with the actor's status in the game engine it will tick all the status per round and thus, can have the effect on the actor who has this effect which use basically all method in ContinuousDamage abstract class implementing Status.
   e. The way polymorphism helps to organize all smoothly and possible.
4. How the classes interact to deliver functionality.

a. The Player uses teleportable object through choosing in console this trigger TeleportAction.
b. TeleportAction then use teleportTo method to handle side effect (e.g., burning surrounding, TeleGround etc... which use SummonFire method burnLocation() etc...) and check if the successful rate succeeds if then move actor to the destination, else randomly choose location on the current map (using RandomLocation class method).
c. The selected teleport object executes its own teleportTo method:
    i. **TeleDoor** → Teleports, burns destination surroundings.
    ii. **TeleportationCircle** → Teleports, burns one random source tile.
    iii. **TeleportCube** → No side effect.
d. If fire is created, a **Fire** tile is placed, lasting 3 turns before turning into **Dirt** this can be done through setGround method.
e. If actor walks on Fire, Fire will check and apply Burning status to the actor.
f. Burning will then use it parent's class ContinuousDamage, applying 5 damage per turn (5 round then remove status from actor).
g. Fire last for 3 turns then set itself to be Dirt.